MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

# DaQuVa: Data Quality Validator.

# DSL

**Mentor:** prof., Vladislav Crucerescu
**Students:** Cimendur Rustem, FAF-243
Frunza Damian, FAF-243
Pancenco Mihail, FAF-243
Tureac Daria, FAF-243

**Chișinău, 2026**

## Abstract

This abstract will be written in the end, after the all other report will be done

# Content

# Introduction

Data cleaning is a fundamental process aimed at detecting and correcting errors, inconsistencies, and redundancies in data to ensure high data quality, especially in large-scale systems such as data warehouses, federated databases, and web-based information platforms. Due to the heterogeneity, volume, and continuous evolution of modern data sources, effective data cleaning requires systematic, scalable, and declarative approaches that minimize manual effort while ensuring correctness and reproducibility [**?** ].

Consequently, there is a growing demand for expressive, scalable, and systematic frameworks that enable users to specify, detect, and repair data quality issues in a declarative and reusable manner. This motivates the development of domain-specific abstractions and languages that can bridge the gap between conceptual data quality requirements and their practical implementation in modern data processing pipelines [**?** **?** ].

Interactive data cleaning systems, such as Potter's Wheel, address the limitations of traditional batch-oriented approaches by tightly integrating discrepancy detection with data transformations. This allows users to iteratively refine cleaning operations on large and heterogeneous datasets through immediate feedback, graphical specifications, and automatic inference of domain structures, reducing the need for complex programming and repeated manual inspection [**?** ].

Wrangler is an interactive system for data transformation that combines direct manipulation of visualized data with automatic inference of applicable transforms. It enables analysts to iteratively clean, reshape, and validate data while capturing an auditable script of all transformations, reducing manual effort and improving reusability [**?** ].

# 1 Domain Analysis

## 1.1 Problem statement

## 1.2 Motivation and need for DSL

## 1.3 Target Audience

## 1.4 Existing solutions analysis

### 1.4.1 Non-DSL solutions

Non-DSL solutions represent the historically dominant way of addressing data quality and data cleaning tasks. Instead of offering a dedicated declarative language, these approaches rely on interactive graphical interfaces, visual specification techniques, enterprise platforms, or direct SQL expressions. As a result, they effectively tackle many practical cleaning scenarios, but they tend to encode quality logic in tools, workflows, or queries rather than in an explicit, reusable language of data quality rules.

**Visual modeling and UML-based quality approaches.**

One direction is to use visual modeling notations such as UML to describe data structures, constraints, and quality requirements on a conceptual level [**?** ]. Such approaches help stakeholders reason about completeness, consistency, and other quality attributes of models, but they do not directly execute validation rules on concrete SQL tables and typically require an additional implementation step in code or database constraints.

**Table 1.4.1 -** Visual UML-based approaches: pros and cons

| **Advantages** | **Limitations** |
|---|---|
| Expressive visual notation for structures and constraints, accessible to architects and analysts. | Models are descriptive only; quality rules are not directly executable on concrete datasets. |
| Facilitates discussion of data quality attributes (completeness, consistency) at design time. | Requires a separate implementation step in SQL, application code, or ETL tools, which can diverge from the model. |
| Supports continuous quality assessment and improvement of models themselves. | Focuses on model-level quality rather than operational validation of large SQL tables in production. |

**Interactive visual data transformation tools (Wrangler).**

Wrangler is an interactive system for specifying data transformation scripts through direct manipulation of tabular data [**?** ]. Analysts iteratively select parts of the dataset, and Wrangler suggests applicable transformations, previews their effects, and records an auditable transformation history [**?** ]. Although Wrangler internally relies on a declarative transformation language, users mainly see a visual interface for shaping and cleaning data, rather than an explicit, reusable language of quality rules.

**Table 1.4.2 -** Wrangler: pros and cons

| Advantages | Limitations |
|---|---|
| Interactive visual interface for specifying complex data transformations without manual scripting. | Focuses on one-off or pipeline-oriented transformations, not on explicit reusable data quality constraints. |
| Mixed-initiative suggestions help users quickly discover useful cleaning operations. | Underlying transformation language is hidden; rules are not exposed as a stable, human-readable contract. |
| Maintains an auditable history of transformations that can be replayed. | Primarily designed for pre-analytics data wrangling rather than continuous validation of SQL tables in production. |

**Interactive and deterministic data repair (Falcon).**

Falcon is an interactive data cleaning system that uses SQL `UPDATE` queries as the underlying repair language and focuses on deterministic, explainable modifications [**?** ]. It encourages users to iteratively explore the space of candidate repairs, presents them as concrete SQL updates, and evaluates their impact on the data [**?** ]. However, Falcon assumes that users are comfortable reasoning about SQL repairs and does not provide a separate, higher-level vocabulary for stating business-level data quality requirements.

**Table 1.4.3 -** Falcon: pros and cons

| Advantages | Limitations |
|---|---|
| Produces deterministic, explainable repairs in the form of explicit SQL `UPDATE` statements. | Assumes users can understand and validate SQL-level repair queries, which may be too low-level for domain experts. |
| Supports interactive exploration of alternative repairs and their impact on the dataset. | Does not define a separate declarative language of data quality rules; repairs are not tied to an explicit rule set. |
| Integrates naturally with relational databases by operating directly on SQL tables. | Focuses on fixing specific instances of dirty data rather than establishing a reusable, auditable quality contract for future data. |

**General-purpose interactive cleaning tools (OpenRefine).**

OpenRefine is a widely used open-source tool for cleaning and transforming tabular data via a spreadsheet-like interface [**?** ]. It supports faceted browsing, clustering similar values, reconciliation with external data sources, and an undo/redoable history of operations that can be replayed on new datasets [**?** ]. OpenRefine is highly effective for one-off or exploratory data cleaning tasks, but its operations are not designed to serve as a formal and reusable contract of data quality for production SQL databases.

**Table 1.4.4 -** OpenRefine: pros and cons

| Advantages | Limitations |
|---|---|
| User-friendly, spreadsheet-like interface suitable for analysts working with messy CSV or JSON data. | Primarily a desktop/GUI tool; not natively integrated as an automated, repeatable check within database pipelines. |
| Powerful features for clustering similar values, faceted filtering, and reconciliation with external sources. | Cleaning operations are sequences of GUI actions or expressions, not explicit, reusable data quality rules over SQL tables. |
| Supports undo/redo and operation history that can be applied to similar datasets. | Better suited for ad-hoc or exploratory cleaning than for serving as a formal data quality contract in production environments. |

**Enterprise data quality platforms.**

Large vendors such as IBM, Oracle, or SAS provide comprehensive data quality platforms that combine profiling, cleansing, standardisation, and matching capabilities [**?** ]. These tools typically integrate tightly with existing ETL and MDM solutions, offer rich dashboards, and support complex matching and deduplication rules for customer or product data. They address enterprise-wide governance requirements but come with significant cost and operational complexity.

**Table 1.4.5 -** Enterprise data quality platforms: pros and cons

| Advantages | Limitations |
|---|---|
| Comprehensive feature set, including profiling, cleansing, matching, and enrichment across many data sources. | High licence and infrastructure costs, typically only justified for large organisations. |
| Tight integration with existing ETL, MDM, and governance tools, enabling end-to-end management of data quality. | Configuration is often complex; setting up rules and workflows requires specialised consultants or trained staff. |
| Rich dashboards and monitoring for continuous tracking of quality metrics at scale. | Business rules are embedded as configuration artefacts inside the platform, not as a lightweight, portable language over SQL tables. |

**SQL-based rule engines and database-centric checks.**

Another class of solutions relies directly on SQL expressions and stored procedures to implement data quality checks and remediation logic [**?** ]. Teams define validation rules as SQL queries or views that flag invalid rows, and sometimes tie them to scheduling and alerting via ETL or orchestration tools. This approach keeps rules close to the database but can become fragmented and hard to audit as the number of checks grows.

**Table 1.4.6 -** SQL-based rule engines: pros and cons

| Advantages | Limitations |
|---|---|
| Uses standard SQL, which is widely known among data engineers and easy to execute in existing databases. | Rules are scattered across queries, views, and stored procedures, making them hard to discover and maintain. |
| Allows fine-grained control and optimisation of checks for specific schemas and performance constraints. | SQL syntax is verbose for complex business rules and not easily readable by non-technical stakeholders. |
| Can be integrated into existing ETL pipelines and CI/CD workflows without introducing new runtime components. | Rules are not expressed as a single, declarative contract; there is no unified catalogue of quality expectations per table. |

**Data profiling and catalog tools with quality rules.**

Data catalog and profiling tools provide automated statistics, distributions, and anomaly detection over database tables and columns. Some of them allow users to attach simple quality rules (for example, thresholds on null rates or uniqueness) and to track violations over time in a central dashboard. They are effective for discovering issues and monitoring trends but typically offer only a limited rule language focused on basic metrics.

**Table 1.4.7 -** Profiling and catalog tools: pros and cons

| Advantages | Limitations |
|---|---|
| Automatically compute descriptive statistics and distributions that reveal hidden data quality problems. | Rule definition is usually restricted to simple thresholds on precomputed metrics (null rate, distinct count, etc.). |
| Centralises quality metrics and rule evaluations in dashboards that are accessible to many stakeholders. | Do not provide an expressive, table-focused language for complex, cross-column business constraints. |
| Help prioritise where to invest cleaning effort by highlighting high-risk tables and columns. | Monitoring is often decoupled from actual repair workflows; suggestions for fixes are weak or absent. |

**Self-service analytics tools with built-in validation.**

Self-service analytics platforms often include basic validation and cleansing functions within their visual workflows. Business users can define filters, simple constraints, and data type checks directly in a drag-and-drop interface and reuse these flows across reports. This reduces the need for ad-hoc scripting but keeps validation logic tied to specific reports rather than to the underlying database tables.

**Table 1.4.8 -** Self-service analytics tools: pros and cons

| Advantages | Limitations |
|---|---|
| Accessible to non-technical users through visual workflows and predefined cleaning components. | Validation rules are often bound to particular reports or dashboards, not to shared database schemas. |
| Enable rapid creation of reusable cleaning and transformation steps inside analytical pipelines. | Rule logic remains hidden inside proprietary workflow definitions, hindering auditing and cross-team reuse. |
| Reduce dependency on engineering teams for basic validation and filtering tasks. | Not designed as a general-purpose data quality layer with an explicit, portable specification of rules. |

**Summary**

Overall, non-DSL solutions demonstrate that interactive visual tools, SQL-based repair systems, enterprise platforms, and profiling tools can substantially reduce manual data cleaning effort and improve transparency of transformations [**? ? ?** ]. At the same time, they either lack a dedicated, human-readable language for expressing reusable data quality rules over SQL tables, or they hide such a language behind configuration and GUIs, which makes it difficult to treat these rules as an explicit, auditable contract between technical and business stakeholders [**? ?** ].

### 1.4.2 Existing DSL solutions

In contrast to non-DSL approaches, a number of research works and industrial systems propose the use of domain-specific languages (DSLs) for data cleaning, transformation, and data quality management. The core idea behind these approaches is to provide a formal, high-level, and declarative way of specifying data processing logic and quality constraints, closely aligned with domain semantics rather than low-level implementation details.

DSL-based solutions typically aim to improve readability, maintainability, and reusability of data cleaning workflows, while enabling explicit and machine-executable specifications of business rules. Empirical studies indicate that the use of DSLs can significantly increase productivity and reduce error rates in complex data manipulation tasks [**?** ]. As a result, DSLs are increasingly explored as a systematic alternative to ad-hoc scripting and purely tool-driven approaches.

One representative example of such a language is **Jayvee** [**?** ], a domain-specific language and execution framework for modeling data processing pipelines together with integrated data quality logic.

Conceptually, Jayvee treats data processing as a formally defined *pipeline model*. A pipeline is a strictly ordered sequence of computational units called *blocks*, where the output of each block becomes the input of the next one. This enforces an explicit, linear dataflow model that makes transformations, dependencies, and execution order unambiguous.

Jayvee defines three fundamental categories of blocks:

- **Extractor blocks** — model data sources (e.g., HTTP endpoints, filesystems, databases) and introduce data into the pipeline.
- **Transformator blocks** — perform structured transformations, filtering, selection, and interpretation of intermediate data representations.
- **Loader blocks** — represent data sinks and persist processed data into storage systems.

This architecture maps directly to the classical ETL (Extract–Transform–Load) paradigm, but formalizes it as a typed, declarative pipeline language rather than as scripts or tool configurations.

A central design element of Jayvee is its *type system based on value types*. Instead of treating data as untyped strings or generic table cells, Jayvee allows users to define domain-specific value types (e.g., percentages, identifiers, measurements) and attach *formal constraints* to them. These constraints define admissible value ranges, formats, and semantic properties, and are evaluated automatically during pipeline execution. As a result, data quality validation becomes an intrinsic part of the execution semantics, not an external checking step.

Jayvee also supports *explicit transformation definitions* that map values between types, enabling controlled and traceable semantic conversions (e.g., unit conversions, format normalization). Furthermore, it allows the definition of *composite block types*, which are higher-level abstractions composed of multiple basic blocks. This enables modularization, reuse, and abstraction of complex processing logic into reusable components.

From a software engineering perspective, Jayvee can be classified as a *typed dataflow DSL* that exhibits:

- an explicit and structured pipeline model,
- formally specified computational units,
- a domain-oriented type system,
- built-in constraint-based data validation, and
- mechanisms for modular composition and reuse.

Consequently, Jayvee transcends traditional transformation scripting and serves as a formal modeling language for data pipelines with integrated data quality semantics.

**Grafter and Grafterizer.**

Another representative DSL-based approach is **Grafter**, together with its interactive environment **Grafterizer** [**?** ]. Grafter is implemented as an embedded domain-specific language (eDSL) in the functional programming language Clojure, targeting data cleaning, transformation, and semantic data publishing workflows. While syntactically realized as a software library, Grafter introduces a dedicated abstraction layer for expressing domain-level data transformation pipelines, thereby fulfilling the defining characteris-

**Table 1.4.9 -** Jayvee: advantages and limitations

| **Advantages** | **Limitations** |
|---|---|
| Formal pipeline model with explicit dataflow semantics and execution order. | Requires learning a new language and modeling paradigm compared to SQL-based solutions. |
| Strong domain-oriented type system with value types and constraints. | Less flexible for highly dynamic or unstructured data compared to script-based approaches. |
| Integrated data quality validation as part of execution semantics. | Primarily focused on structured pipelines; not designed for ad-hoc exploratory cleaning. |
| Clear separation of extraction, transformation, and loading stages. | Depends on predefined block types for integration with external systems. |
| Support for modular composition via composite blocks and reusable components. | Smaller ecosystem compared to mature enterprise ETL platforms. |
| Declarative, readable specification of complex data workflows. | Requires formal modeling effort even for simple pipelines. |

tics of a DSL.

Conceptually, Grafter follows a *functional pipeline model*, where data transformations are represented as composable functions, called *pipes*. Each pipe performs a single, well-defined transformation step, and complex workflows are constructed through functional composition. This model enforces a clear and explicit dataflow structure, in which intermediate results are immutable and transformation semantics are deterministic, enabling transparent reasoning about pipeline behavior.

A distinctive feature of Grafter is its strong integration of *data cleaning and semantic transformation*. In addition to conventional tabular cleaning operations such as filtering, normalization, enrichment, and value derivation, Grafter provides first-class constructs for mapping cleaned data into RDF representations. Users define semantic mappings using explicit triple generation rules, enabling the transformation of raw tabular data into Linked Data formats suitable for publication and semantic interoperability.

Grafterizer extends the Grafter DSL by providing an interactive graphical environment for constructing, testing, and executing transformation pipelines. Through a visual interface, users can compose pipelines from predefined transformation operators, preview intermediate results, and iteratively refine workflows. This interactive layer lowers the barrier of entry for domain experts while preserving the expressive power and compositional semantics of the underlying DSL.

From a software engineering perspective, Grafter can be classified as a *functional embedded DSL* characterized by:

- compositional transformation pipelines,
- functional abstraction and immutability,
- explicit dataflow modeling,
- extensible operator libraries, and

- integrated semantic data mapping.

This design makes Grafter particularly suitable for large-scale data transformation and semantic publishing workflows, while Grafterizer facilitates interactive development and validation of data cleaning logic.

**Table 1.4.10 -** Grafter and Grafterizer: advantages and limitations

| **Advantages** | **Limitations** |
|---|---|
| Functional pipeline DSL enables modular, compositional data transformations. | Requires understanding of functional programming concepts, which may be unfamiliar to many data analysts. |
| Integrated support for both data cleaning and RDF generation within a single language framework. | Strong orientation toward semantic web workflows limits applicability to purely relational or analytical pipelines. |
| Clear and explicit dataflow semantics improve reasoning about transformation correctness. | Embedded DSL syntax inherits complexity from the host language (Clojure). |
| Interactive graphical interface supports incremental pipeline development and debugging. | Graphical abstraction may obscure underlying transformation semantics for advanced users. |
| Extensible through custom transformation operators and semantic mappings. | Custom extensions require proficiency in Clojure and JVM tooling. |
| Supports semantic interoperability and Linked Data publishing. | More complex deployment and runtime environment than pure SQL-based validation solutions. |

**Wrangle (Trifacta / Alteryx).**

Wrangle is a proprietary domain-specific language used internally by the Trifacta (now Alteryx Designer Cloud) platform to specify data transformation recipes [**?** ]. Although users primarily interact with Wrangle through a graphical interface, all transformation logic is ultimately compiled into a sequence of formal Wrangle expressions. As such, Wrangle represents a *hidden DSL*, whose syntax and semantics are abstracted away from direct user interaction.

Conceptually, Wrangle follows a *stepwise transformation model*, where a dataset is transformed through an ordered sequence of transformation steps called *recipes*. Each recipe step corresponds to a formal transformation operator (transform) applied to selected columns and rows, optionally parameterized by expressions, functions, and predicates. This structure defines a linear dataflow pipeline, in which each step incrementally modifies the dataset state.

The Wrangle language provides a rich collection of built-in operators and functions covering string processing, numerical computation, date handling, statistical aggregation, window operations, and nested data manipulation. Transformation expressions combine column references, functional operators, and logical predicates, enabling concise yet expressive specification of complex cleaning operations. However, direct authoring of Wrangle scripts is intentionally restricted; instead, users construct recipes via interactive selection and manipulation, with the system generating the corresponding DSL code implicitly.

From a software engineering perspective, Wrangle can be classified as a *proprietary embedded DSL for data transformation pipelines* characterized by:

- linear recipe-based pipeline structure,
- operator-centric transformation semantics,
- expression-based column and row selection,
- integrated functional computation model, and
- tight coupling to a graphical interaction environment.

While this design significantly lowers the entry barrier for non-technical users, it also limits transparency, portability, and independent reuse of transformation logic outside the hosting platform.

**Table 1.4.11 -** Wrangle DSL: advantages and limitations

| **Advantages** | **Limitations** |
|---|---|
| Rich set of built-in transformation operators and functions for diverse cleaning tasks. | Proprietary and closed language, not independently reusable outside the Alteryx ecosystem. |
| Interactive construction of pipelines lowers the entry barrier for non-technical users. | Transformation logic remains hidden, reducing transparency and auditability. |
| Formal underlying DSL enables deterministic and reproducible execution of workflows. | Users cannot directly author or maintain Wrangle code as a first-class artifact. |
| Supports complex nested, statistical, and window-based transformations. | Strong vendor lock-in; workflows are not portable across systems. |
| Seamless integration with enterprise analytics and cloud execution environments. | Primarily designed for interactive analytics, not for explicit specification of reusable data quality contracts. |

## 1.5 Summary of Existing Solutions

Existing solutions for data cleaning and data quality management can be broadly classified into **non-DSL approaches** and **DSL-based approaches**.

Non-DSL approaches, including visual modeling (e.g., UML-based quality assessment), interactive data transformation tools (e.g., Wrangler, Falcon, OpenRefine), enterprise platforms (e.g., IBM, Oracle, SAS), and SQL-based rule engines, primarily rely on either graphical interfaces, ad-hoc scripts, or embedded configurations to perform data cleaning and validation [**? ? ? ? ?** ]. These solutions offer several practical advantages:

- Facilitate iterative and interactive cleaning processes.
- Reduce manual effort for common data transformations.
- Integrate with existing ETL pipelines and enterprise workflows.
- Provide audit trails or histories of applied transformations.

However, they also exhibit significant limitations:

- Data quality rules are often hidden, ad-hoc, or fragmented across queries, workflows, or GUI actions.
- Lack of a formal, human-readable, and reusable specification of business-level quality rules.
- Limited support for automated validation and modular composition of rules.

DSL-based approaches, such as Jayvee [**?** ] and Grafter/Grafterizer [**?** ], introduce domain-specific abstractions for modeling pipelines and composable transformations. They formalize aspects of data processing through explicit constructs, typed value systems, and built-in validation mechanisms. The advantages of DSL-based solutions include:

- Explicit and structured representation of transformations and dataflow.
- Integrated constraint-based validation and semantic reasoning.
- Support for modular composition and reuse of processing units.

Their limitations include:

- Steeper learning curve due to new languages and paradigms.
- Focus on transformation pipelines rather than purely on data quality contracts.
- Smaller ecosystems compared to mature enterprise ETL platforms.

Overall, while existing solutions reduce manual data cleaning effort and increase transparency of transformations, they generally do not provide a dedicated, formal language for specifying *auditable, reusable, and declarative data quality rules* over relational tables. This gap motivates the development of a dedicated DSL for data quality validation.

### 1.6 Identified Gaps

Despite the diversity of existing solutions for data cleaning and data quality management, several limitations remain that motivate the development of a dedicated domain-specific language (DSL):

1. **Lack of explicit, reusable rules:** Non-DSL tools often embed validation logic in GUI actions, scripts, or platform configurations, making it difficult to extract, audit, and reuse business-level data quality rules.
2. **Fragmented validation:** SQL-based and enterprise approaches frequently scatter checks across multiple queries, stored procedures, or pipelines, complicating maintenance, discoverability, and traceability.
3. **Limited formal semantics:** Most visual or script-driven solutions do not provide formal semantics for transformations or data constraints, which hinders automated reasoning, error detection, and reproducibility.
4. **Low modularity and composability:** Existing solutions often lack a mechanism for defining modular, composable transformations or quality rules that can be reused across datasets and projects.
5. **High dependence on technical expertise:** Tools such as Falcon, Wrangler, and Grafter assume familiarity with SQL, functional programming, or pipeline modeling, which can exclude domain experts

from participating directly in quality specification.

6. **Insufficient integration of semantic validation:** Only a few DSL-based approaches (e.g., Grafter) combine data cleaning with semantic or type-based validation, leaving a gap in end-to-end formalized data quality enforcement.
7. **Steep learning curves and ecosystem limitations:** DSL-based solutions often require adoption of new languages and paradigms, and their smaller ecosystems make integration, tooling, and community support limited compared to mature enterprise platforms.

These gaps indicate that while existing approaches improve efficiency and provide partial solutions, there is no unified, formal, and accessible mechanism to specify, enforce, and audit data quality rules in a reusable and domain-oriented manner. Addressing these gaps motivates the design of a dedicated *Data Quality Validator DSL*, which aims to combine the formal rigor of DSL-based solutions with the usability and accessibility required for widespread adoption by both technical and domain experts.

## 1.7 Functional Requirements

The Data Quality Validator DSL aims to provide a formal, expressive, and user-friendly language for specifying, validating, and enforcing data quality rules. The functional requirements of the DSL can be grouped into the following categories:

### 1.7.1 Rule Definition and Specification

- **Declarative rule syntax:** Users should be able to express data quality rules in a clear, human-readable, and declarative form.
- **Support for multiple data types:** The language must allow rules over standard types such as integers, decimals, strings, dates, and domain-specific types (e.g., percentages, IDs).
- **Constraint specification:** Users can define constraints on values, ranges, formats, uniqueness, relationships between columns, and custom predicates.
- **Composite rules:** The DSL should support composing multiple simple rules into complex, reusable rulesets.

### 1.7.2 Validation and Execution

- **Table-level validation:** The DSL must allow rules to be applied over entire tables or datasets.
- **Row-level and column-level checks:** Users can define rules that operate at granular levels (per row, per column, or across multiple columns).
- **Deterministic evaluation:** Rule execution should produce deterministic results, ensuring reproducibility.
- **Error reporting and diagnostics:** The system must provide clear feedback on violations, including location (row/column) and the rule that failed.

- **Batch and incremental validation:** The DSL should support validating historical datasets as well as streaming or newly appended data.

#### 1.7.3 Modularity and Reusability

- **Reusable rule libraries:** Users should be able to define, store, and reuse commonly applied rules across datasets and projects.
- **Parameterized rules:** The language should allow parameterization to apply the same logic to different columns, tables, or datasets without rewriting rules.
- **Rule grouping and categorization:** Rules can be grouped by domain, dataset, or quality attribute for easier management and auditing.

#### 1.7.4 Integration and Extensibility

- **Database connectivity:** The DSL must support reading from and writing to relational databases (e.g., SQL-based systems) and potentially flat files (CSV, JSON).
- **Extensible functions and predicates:** Users can define custom functions or predicates to handle specialized validation logic.
- **Interoperability:** Rulesets can be exported, imported, or integrated with existing ETL pipelines and monitoring systems.

#### 1.7.5 Usability and Accessibility

- **Readable syntax:** Rule expressions should be intuitive to domain experts with minimal programming background.
- **Interactive development support:** Optional support for IDE-like features, auto-completion, and inline validation to facilitate rule creation.
- **Auditability:** All rule definitions, executions, and violations should be logged for auditing and compliance purposes.

In summary, the Data Quality Validator DSL must combine *formal rigor*, *reusable abstractions*, and *user accessibility*, addressing the limitations of both non-DSL and existing DSL solutions, while enabling automated, deterministic, and auditable enforcement of data quality rules.

### 1.8 Proposed Solution Overview

The proposed Data Quality Validator DSL is a declarative language designed to define, execute, and manage data quality rules efficiently. It allows users to write high-level scripts that specify constraints, validations, and transformations on datasets in a human-readable and reusable format.

**Example: Column-level validation**

The following DSL snippet ensures that the ‘age‘ column contains only positive integers:

```
validate column "age" {
    type: integer
```

```
    condition: value > 0
    on_error: "Age must be positive"
}
```

**Example: Cross-column rule**

The next snippet ensures that ‘end$_d$ate‘isalwayslaterthan‘start$_d$ate‘ :

```
validate row {
    condition: end_date > start_date
    on_error: "End date must be after start date"
}
```

**Example: Referential integrity check**

This example enforces that ‘customer$_i$d‘existsinthe‘customers‘table :

```
validate column "customer_id" {
    reference: "customers.id"
    on_error: "Customer ID not found"
}
```

**Example: Reusable rule definition**

Rules can be parameterized and reused across datasets:

```
rule positive_number(column_name) {
    validate column column_name {
        type: integer
        condition: value > 0
        on_error: "Value must be positive"
    }
}


apply positive_number("age")
apply positive_number("salary")
```

These examples demonstrate the DSL’s **readability, reusability, and expressiveness**, allowing both domain experts and engineers to define data quality rules without low-level SQL or procedural code.

### 1.9 Use Cases

The DSL supports a variety of practical scenarios:

**1. Batch validation of datasets**

```
dataset "employees.csv" {
```

```
    validate column "email" {
        pattern: /^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/
        on_error: "Invalid email format"
    }
    validate column "salary" {
        condition: value >= 0
        on_error: "Salary must be non-negative"
    }
}
```

**2. Incremental validation for streaming data**

```
stream "transactions" {
    validate row {
        condition: amount > 0
        on_error: "Transaction amount must be positive"
    }
}
```

**3. Aggregated checks and group-level rules**

```
validate group "orders" by "customer_id" {
    condition: sum(amount) <= credit_limit
    on_error: "Customer exceeded credit limit"
}
```

**4. Logging and auditing**

```
log "validation_report.log" {
    include: all_errors
    format: csv
}
```

These use cases illustrate how the DSL can be applied to column-level, cross-column, group-level, and streaming validations, while supporting modular, auditable, and reusable rule definitions. Users can write scripts like these to enforce consistent and high-quality datasets across multiple projects.

# 2 DaQuVa DSL: Language Design

## 2.1 Introduction to DaQuVa DSL Design

DaQuVa (Data Quality Validator) is a domain-specific language designed to facilitate the safe and efficient cleaning, validation, and analysis of large datasets within SQL databases. The language adopts a declarative programming paradigm, emphasizing ”what” to do rather than ”how” to do it, which aligns with the needs of data engineers and analysts who require concise, readable scripts for data quality tasks.

The DSL is tailored for post-validation data cleaning in enormous databases, where data has already undergone initial validation but may still contain inconsistencies, duplicates, or errors. DaQuVa ensures cautious operations by incorporating confidence-based scoring (e.g., dirty_level, duplicate_score) and explicit safety mechanisms (e.g., Here() with allowDanger) to prevent accidental data loss.

Key design principles include:

- **Declarative Nature**: Users describe data transformations and validations without specifying low-level implementation details.
- **Safety-First**: Destructive operations require explicit thresholds and permissions.
- **Reusability**: Support for variables and functions to enable modular, pipeline-friendly scripts.
- **Extensibility**: Architecture allows integration of AI-based or rule-based tools for validation.

## 2.2 Initial DSL Grammar

### 2.2.1 Design Principles

The grammar of DaQuVa is guided by three principles:

1. **Readability.** Statements read like short English instructions (`scan table."col" using tool "name"`).
2. **Minimalism.** Only constructs required for data-quality tasks are included; general-purpose programming facilities are deliberately omitted.
3. **Safety by default.** Destructive operations on the underlying database require an explicit `allowDanger` flag, making accidental data modification syntactically impossible.

### 2.2.2 Lexical Elements

The lexical layer defines the primitive tokens consumed by the parser.

**Keywords**

The reserved words of DaQuVa are grouped by functional category in Table 2.2.1. They may not be used as identifiers.

**Table 2.2.1 -** Reserved keywords of DaQuVa, grouped by category.

| **Category** | **Keywords** |
|---|---|
| Database | `my_db` |
| Function & Control | `function`, `return` |
| Commands | `scan`, `filter`, `merge`, `output`, `newTable`, `findLocalDuplicates` |
| Conditions & Logic | `where`, `AND`, `OR`, `columns` |
| Tool Integration | `using`, `tool` |
| Output Targets | `console`, `file`, `from`, `addColumns` |
| Safety Flags | `Here`, `allowDanger` |

**Identifiers**

An identifier begins with a letter and may be followed by any combination of letters, digits, and underscores:

```
<identifier> ::= <letter> { <letter> | <digit> | "_" }
<letter>     ::= "a" | ... | "z" | "A" | ... | "Z"
<digit>      ::= "0" | ... | "9"
```

**Literals**

DaQuVa supports two literal types:

```
<string> ::= '"' { <any_char_except_quote> } '"'
<number> ::= <digit> { <digit> }
```

`<number>` denotes a non-negative integer, used for thresholds and confidence scores. String literals are delimited by double quotes and may contain any character except an unescaped double quote.

**Comments**

Single-line comments begin with # and extend to the end of the line. Comments are ignored by the parser.

**Whitespace and Layout**

Unlike indentation-sensitive languages such as Python, DaQuVa treats whitespace purely as a lexical separator. The parser ignores formatting characters and does not assign semantic meaning to indentation.

The following rules apply:

- One or more whitespace characters may appear between tokens to separate them.
- Consecutive spaces are treated as a single separator.
- Tab characters and newline characters are ignored by the parser.

- Program structure is defined by explicit punctuation rather than indentation.
- The semicolon (;) is the only required statement terminator and separates commands.

As a result, developers may freely format DaQuVa programs for readability. The following examples are syntactically equivalent:

```
filter potential_dirty where dirty_level > 70;
```

```
filter     potential_dirty
    where    dirty_level    >     70    ;
```

This design follows the convention of languages such as C and SQL, where whitespace improves readability but does not influence program semantics.

**Operators and Punctuation**

Table 2.3.1 lists the core semantic entities of DaQuVa.

**Table 2.2.2 -** Core semantic entities of DaQuVa.

| Entity | Description |
|---|---|
| `my_db` | Declares the active database connection using an opaque connection string. Exactly one `my_db` declaration is expected per program; re-declaration overwrites the current context. |
| Table | The primary data container. Tables exist either in the connected database or as in-memory result sets produced by `scan` or `newTable`. |
| Variable | Any command result can be bound to a variable. Variables are untyped at the syntax level but carry a *result-set* value at runtime. |
| Tool | An external validator or AI-powered analyser identified by a string name (e.g., `"fuzzy_matcher"`, `"typo_checker"`). The runtime resolves the name to a registered tool implementation. |
| Column / Meta-variable | Columns such as `dirty_level` and `duplicate_score` are injected into result sets automatically by `scan` and `findLocalDuplicates`, respectively. They may also be added explicitly via `addColumns`. |
| Function | A named, reusable sequence of commands parameterised by identifiers. Calling a function executes its body in a new scope; parameters shadow outer variables. |
| Threshold / Confidence | An integer in the range $[0, 100]$ representing a minimum quality score. Used in `filter` conditions and as the confidence argument to `Here()`. |
| `Here()` | A special sink that redirects writes into the live database. Requires an optional confidence argument; when used with `merge`, it writes merged records back to the source. |
| `allowDanger` | A boolean flag that must be present on any `merge` command that targets `Here()`. It serves as an explicit acknowledgement that the operation modifies persisted data. |

### 2.2.3 Syntactic Grammar in BNF

The complete context-free grammar of DaQuVa is given below. Non-terminals are enclosed in angle brackets; terminals are quoted. Curly braces denote zero-or-more repetition; square brackets denote an optional element.

Listing 2.2.1 - Complete BNF grammar of DaQuVa.

```
<program> ::= <statement> <program>
```

```
            | <empty>

<empty> ::=


<statement> ::= <db_init>
              | <variable_assign>
              | <function_def>
              | <command_statement>


<db_init> ::= "my_db" <string> ";"


<variable_assign> ::= <identifier> "=" <command_statement> ";"


<function_def> ::= "function" <identifier>
                   "(" <opt_param_list> ")" ":"
                   <statement_block>


<opt_param_list> ::= <param_list>
                   | <empty>


<param_list> ::= <identifier> <param_list_tail>

<param_list_tail> ::= "," <identifier> <param_list_tail>
                    | <empty>


<statement_block> ::= <statement> <statement_block>
```

```
                    | <empty>

<command_statement> ::= <scan_command>
                      | <filter_command>
                      | <find_duplicates_command>
                      | <merge_command>
                      | <output_command>
                      | <new_table_command>
                      | <function_call>

<scan_command> ::= "scan" <table_column> "using" "tool" <tool_name>

<tool_name> ::= <string>

<table_column> ::= <table_name> "." <column_name>

<table_name> ::= <string>

<column_name> ::= <string>

<filter_command> ::= "filter" <identifier> "where" <condition>

<condition> ::= <expression> <condition_tail>

<condition_tail> ::= <logical_op> <expression> <condition_tail>
                   | <empty>
```

```
<logical_op> ::= "AND"
               | "OR"

<expression> ::= <identifier> <comparison_op> <number>

<comparison_op> ::= ">"
                  | "<"
                  | ">="
                  | "<="
                  | "=="
                  | "!="

<find_duplicates_command> ::= "findLocalDuplicates" <identifier>
                              "columns" "[" <column_list> "]"
                              <opt_using_tool>

<opt_using_tool> ::= "using" "tool" <string>
                   | <empty>

<column_list> ::= <string> <column_list_tail>

<column_list_tail> ::= "," <string> <column_list_tail>
                     | <empty>

<merge_command> ::= "merge" <identifier>
```

```
                    "->" "Here" <opt_merge_offset> <opt_allow_danger>

<opt_merge_offset> ::= "(" <number> ")"
                     | <empty>

<opt_allow_danger> ::= "allowDanger"
                     | <empty>

<output_command> ::= "output" <identifier> "->" <output_target>

<output_target> ::= "console"
                  | "file" <string>
                  | "Here" <opt_output_offset>

<opt_output_offset> ::= "(" <number> ")"
                      | <empty>

<new_table_command> ::= "newTable" <string>
                        <opt_from>
                        <opt_add_columns>

<opt_from> ::= "from" <identifier>
             | <empty>

<opt_add_columns> ::= "addColumns" "[" <column_func_list> "]"
```

```
                    | <empty>

<column_func_list> ::= <column_func> <column_func_list_tail>

<column_func_list_tail> ::= "," <column_func> <column_func_list_tail>
                          | <empty>

<column_func> ::= <string>
                | <func_call>

<func_call> ::= <identifier> "(" <opt_arg_list> ")"

<function_call> ::= <identifier> "(" <opt_arg_list> ")"

<opt_arg_list> ::= <arg_list>
                 | <empty>

<arg_list> ::= <value> <arg_list_tail>

<arg_list_tail> ::= "," <value> <arg_list_tail>
                  | <empty>

<value> ::= <identifier>
          | <string>
          | <number>
```

```
<identifier> ::= <letter> <identifier_tail>

<identifier_tail> ::= <letter> <identifier_tail>
                    | <digit> <identifier_tail>
                    | "_" <identifier_tail>
                    | <empty>


<number> ::= <digit> <number_tail>

<number_tail> ::= <digit> <number_tail>
                | <empty>


<string> ::= '"' <string_body> '"'

<string_body> ::= <any_char_except_quote> <string_body>
                | <empty>
```

### 2.2.4 Core Syntactic Structures

**Program and Statements**

A DaQuVa program is a flat sequence of statements with no required entry-point. Execution proceeds top-to-bottom. There are four statement kinds:

- **Database initialisation** (`my_db`): establishes the active connection.
- **Variable assignment**: binds the result of a command to a name.
- **Function definition**: introduces a named, reusable routine.
- **Command statement**: executes an action without binding.

**Expressions and Conditions**

Conditions used in `filter` commands consist of one or more *expressions* joined by `AND` / `OR`. Each expression compares a column-level attribute (e.g. `dirty_level`) to an integer threshold using a standard comparison operator. This deliberately limited expression language prevents arbitrary logic and keeps programs auditable.

#### Functions

Functions accept a comma-separated list of positional parameters and contain an indentation-delimited block of statements. Unlike general-purpose languages, DaQuVa functions may only contain the same statement types available at the top level; there is no nested function definition or recursion.

## 2.3 Basic DSL Semantics

### 2.3.1 Execution Model

DaQuVa programs execute *sequentially*. Each statement is fully evaluated before the next begins. The runtime maintains:

1. A single **database context** set by `my_db`.
2. A **variable environment** mapping names to result sets.
3. A **function table** mapping names to parameter lists and statement blocks.

There is no implicit concurrency, no exception-handling syntax, and no dynamic typing — every variable holds a *result set* (a typed, annotated table-like structure).

### 2.3.2 Semantic Description of Core Constructs

Table 2.3.1 summarises the principal semantic entities of DaQuVa.

**Table 2.3.1 -** Core semantic entities of DaQuVa

| Entity | Description |
|---|---|
| `my_db` | Declares the active database connection using an opaque connection string. Exactly one `my_db` declaration is expected per program; re-declaration overwrites the current context. |
| Table | The primary data container. Tables exist either in the connected database or as in-memory result sets produced by `scan` or `newTable`. |
| Variable | Any command result can be bound to a variable. Variables are untyped at the syntax level but carry a *result-set* value at runtime. |
| Tool | An external validator or AI-powered analyser identified by a string name (e.g. `"fuzzy_matcher"`, `"typo_checker"`). The runtime resolves the name to a registered tool implementation. |
| Column / Meta-variable | Columns such as `dirty_level` and `duplicate_score` are injected into result sets automatically by `scan` and `findLocalDuplicates`, respectively. They may also be added explicitly via `addColumns`. |
| Function | A named, reusable sequence of commands parameterised by identifiers. Calling a function executes its body in a new scope; parameters shadow outer variables. |
| Threshold / Confidence | An integer in the range $[0, 100]$ representing a minimum quality score. Used in `filter` conditions and as the confidence argument to `Here()`. |
| `Here()` | A special sink that redirects writes into the live database. Requires an optional confidence argument; when used with `merge`, it writes merged records back to the source. |
| `allowDanger` | A boolean flag that must be present on any `merge` command that targets `Here()`. It serves as an explicit acknowledgement that the operation modifies persisted data. |

### 2.3.3 Command Semantics

#### `scan` — Column Validation

<scan_command> ::= "scan" <table_column> "using" "tool" <string>

**Meaning.** `scan` passes every value in the named column of the named table to the specified tool. The tool assigns each row a `dirty_level` score in $[0, 100]$, where 100 represents maximum dirtiness. The result is an annotated copy of the rows with the added meta-column.

**Execution.**

1. Resolve the table name against the active database context.

2. Resolve the tool name against the registered tool registry.
3. Invoke the tool for each row value in the column.
4. Return a result set containing all original columns plus `dirty_level`.

**Errors.** An error is raised if the table or column does not exist, if the tool name cannot be resolved, or if the database context has not been initialised.

`filter` **— Threshold Filtering**

```
<filter_command> ::= "filter" <identifier> "where" <condition>
```

**Meaning.** Produces a subset of the input result set containing only rows that satisfy the given condition.

**Execution.** Each row is evaluated against the condition; rows for which the condition is true are retained. Conditions may combine multiple predicates with `AND` / `OR`; evaluation follows standard boolean semantics with left-to-right associativity.

**Errors.** An error is raised if the referenced variable is undefined, if a column referenced in a condition does not exist in the result set, or if operand types are incompatible.

`findLocalDuplicates` **— Duplicate Detection**

```
<find_duplicates_command>
    ::= "findLocalDuplicates" <identifier>
            "columns" "[" <column_list> "]"
            [ "using" "tool" <string> ]
```

**Meaning.** Identifies rows within the input result set that are potential duplicates according to the specified key columns. An optional AI-powered tool (e.g. a fuzzy matcher) provides similarity scoring.

**Execution.** Rows are grouped by the specified columns; within each group, pairs are scored for similarity. The result set contains all candidate duplicate rows annotated with a `duplicate_score` in $[0, 100]$.

**Errors.** An error is raised if the input variable is undefined or if any listed column does not exist in the result set.

`merge` **— Duplicate Resolution**

```
<merge_command> ::= "merge" <identifier>
                            "->" "Here" [ "(" <number> ")" ]
                            [ "allowDanger" ]
```

**Meaning.** Merges duplicate records in the input result set and writes the resolved records back to the live database through `Here()`. The optional numeric argument specifies a minimum confidence threshold; only duplicate pairs with `duplicate_score` $\geq$ threshold are merged.

**Execution.**

1. Filter candidates by the confidence threshold (default: 0).
2. For each qualifying pair, apply a deterministic merge strategy (e.g. keep the most complete record).
3. Write merged records to the database; delete superseded records.

**Safety constraint.** The `allowDanger` flag *must* be present. Its absence is a compile-time error. This design makes destructive database modifications opt-in and visible in code review.

`output` **— Result Export**

```
<output_command> ::= "output" <identifier> "->" <output_target>
<output_target>  ::= "console" | "file" <string> | "Here" [ "(" <number
    > ")" ]
```

**Meaning.** Writes a result set to one of three sinks: the console (standard output), a named file, or the live database via `Here()`.

**Execution.**

- `console` — serialises rows as human-readable text.
- `file "<path>"` — writes rows to the specified file path in a delimited format (e.g. CSV).
- `Here()` — inserts or upserts rows into the database.

**Errors.** An error is raised if the variable is undefined, if the file path is not writable, or if the database context is missing for `Here()`.

`newTable` **— Table Construction**

```
<new_table_command> ::= "newTable" <string>
                                [ "from" <identifier> ]
                                [ "addColumns" "[" <column_func_list> "]" ]
```

**Meaning.** Creates a new named table, optionally seeded from an existing result set and optionally extended with computed columns.

**Execution.** If `from` is provided, the new table is initialised with the rows of the referenced result set. Each entry in `addColumns` is either a plain column name (copied from the source) or a function call whose result becomes a new column value (e.g. `length("name")`).

`function` **and Function Calls**

```
<function_def>  ::= "function" <identifier> "(" [ <param_list> ] ")"
    ":"
                                <statement_block>
<function_call> ::= <identifier> "(" [ <arg_list> ] ")"
```

**Meaning.** Functions encapsulate reusable command sequences. They are *first-defined* (must be defined before use in a top-to-bottom interpreter) and execute in their own variable scope. Parameters are positional; the number of arguments at the call site must match the parameter list.

**Errors.** Arity mismatch, calling an undefined function, or a `return` statement referencing an undefined variable all raise errors.

### 2.3.4 Error Catalogue

Table 2.3.2 catalogues the principal errors and their triggers.

**Table 2.3.2 -** Error catalogue for DaQuVa.

| Error | Condition |
|---|---|
| UndefinedVariable | An identifier is used before assignment. |
| UndefinedTable | A table name cannot be resolved in the database context. |
| UndefinedColumn | A column referenced in a condition or column list does not exist in the result set. |
| UnresolvedTool | A tool string cannot be matched to a registered tool. |
| ArityMismatch | A function call provides more or fewer arguments than parameters declared. |
| MissingContext | A database-dependent command is issued before `my_db`. |
| MissingAllowDanger | A `merge -> Here()` command lacks the `allowDanger` flag. |
| InvalidThreshold | A numeric argument to `Here()` or a filter expression falls outside $[0, 100]$. |
| IOError | The file path in an `output -> file` command is not accessible. |

## 2.4 Illustrative Example Program

The program below demonstrates all major language constructs. It connects to a database, defines a reusable email-cleaning function, applies it, exports results, detects and merges duplicates, and creates a summary table for further analysis.

Listing 2.4.1 - Annotated DaQuVa example program.

```
# === 1. Database connection ===
my_db "some_connection_string_goes_here";

# === 2. Reusable function: clean emails in any table ===
function cleanEmails(table_name, threshold):
    # Scan the email column with an AI validator
```

```
    potential_dirty = scan table_name."email" using tool "
       misa_ai_validator";

    # Keep only rows whose dirty_level exceeds the threshold
    high_risk = filter potential_dirty where dirty_level > threshold;

    return high_risk;

# === 3. Apply the function to the "users" table ===
high_risk_emails = cleanEmails("users", 70);

# === 4. Export results ===
output high_risk_emails -> file "high_risk_emails.csv";
output high_risk_emails -> console;

# === 5. Detect near-duplicate records among high-risk rows ===
duplicates = findLocalDuplicates high_risk_emails
             columns ["name", "dob"]
             using tool "fuzzy_matcher";

# === 6. Merge duplicates back into the database (confidence >= 90%)
   ===
merge duplicates -> Here(90) allowDanger;

# === 7. Create a summary table for downstream analysis ===
analysis_table = newTable "high_risk_analysis"
                 from high_risk_emails
                 addColumns [duplicate_score, length("name")];

# === 8. Additional validation pass on the summary table ===
cli_result = scan analysis_table."email" using tool "typo_checker";
output cli_result -> console;
```

The program illustrates several semantic properties:

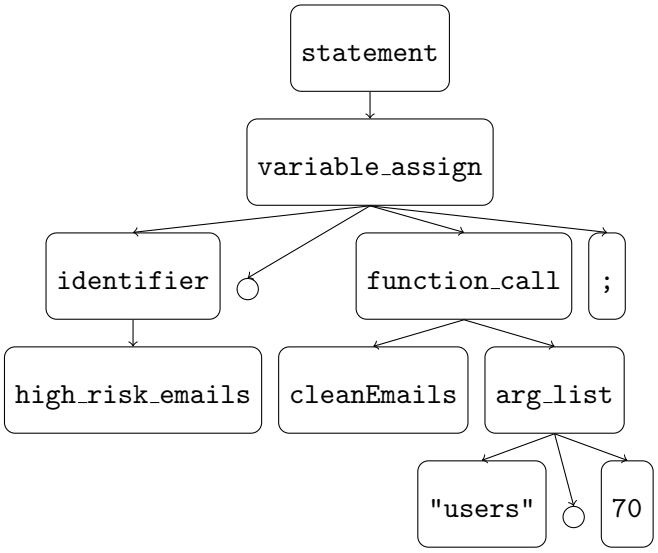
- The function `cleanEmails` abstracts a two-step scan-then-filter pipeline, making the threshold a tuneable parameter.
- The `output` command is used twice on the same variable, demonstrating that result sets are non-destructively consumed.
- The `merge` command requires `allowDanger` because it writes to the live database, making the risky operation immediately visible to reviewers.
- `newTable` constructs a derived table that mixes copied data with computed columns (`duplicate_score`, `length("name")`), enabling further analysis without modifying the source table.

## 2.5 Derivation Trees for Control Structures

Below are the derivation trees for the main control structures of DaQuVa.

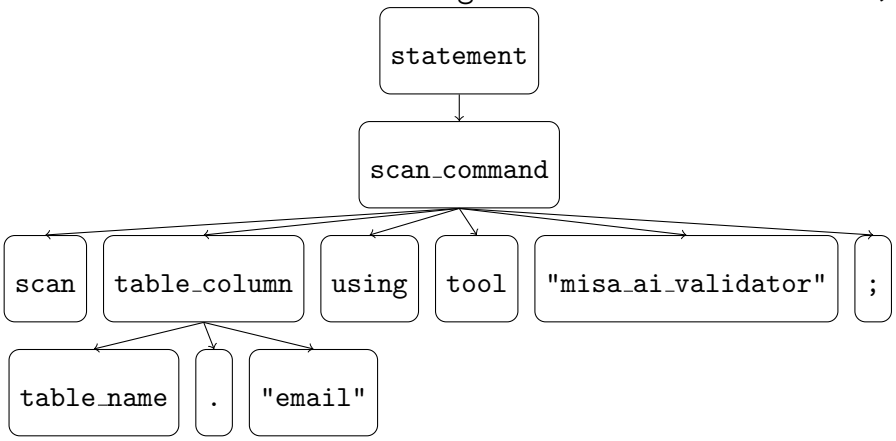
### 2.5.1 Variable Definition

**Example:** `high_risk_emails = cleanEmails("users", 70);`



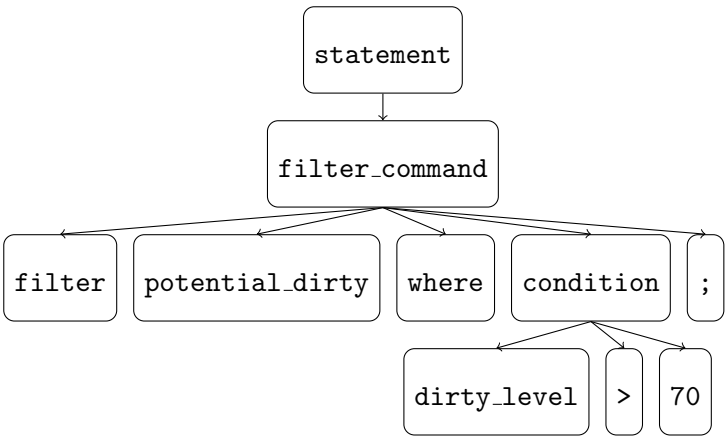

### 2.5.2 scan Command

**Example:** `scan table_name."email" using tool "misa_ai_validator";`



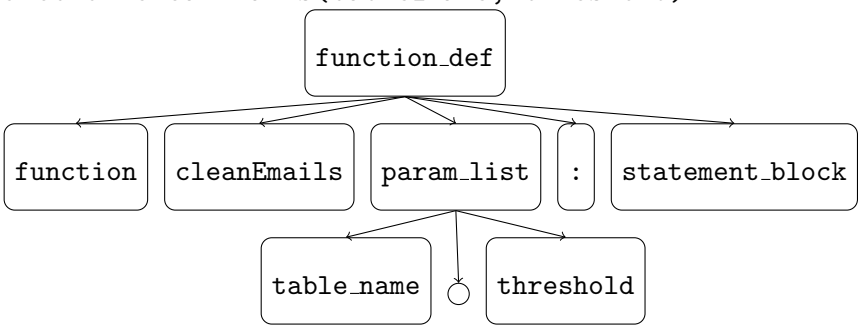

### 2.5.3 filter Command

**Example:** `filter potential_dirty where dirty_level > 70;`



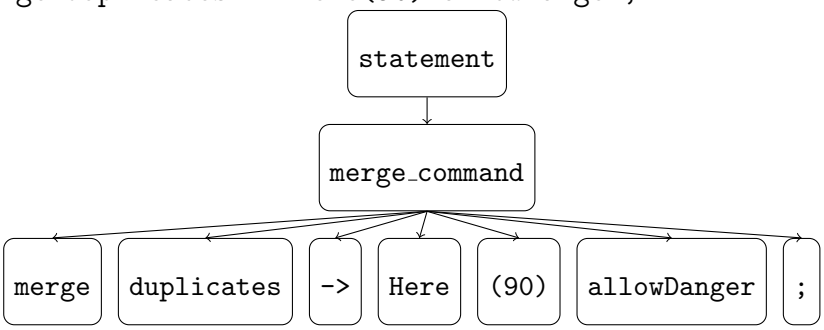

### 2.5.4 Function Definition

**Example:** `function cleanEmails(table_name, threshold):`



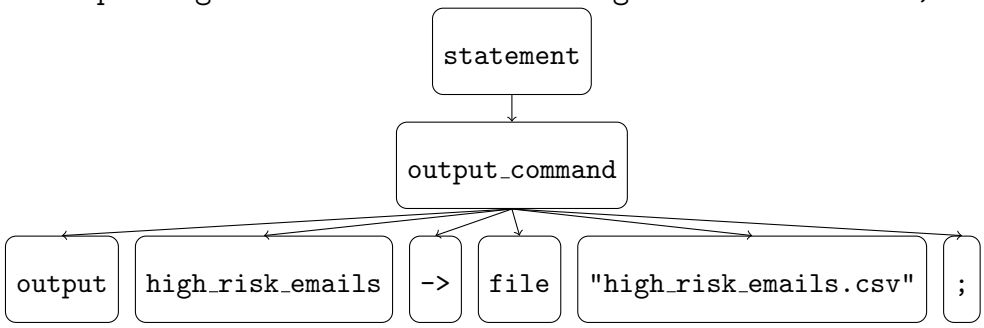

### 2.5.5 merge Command

**Example:** `merge duplicates -> Here(90) allowDanger;`



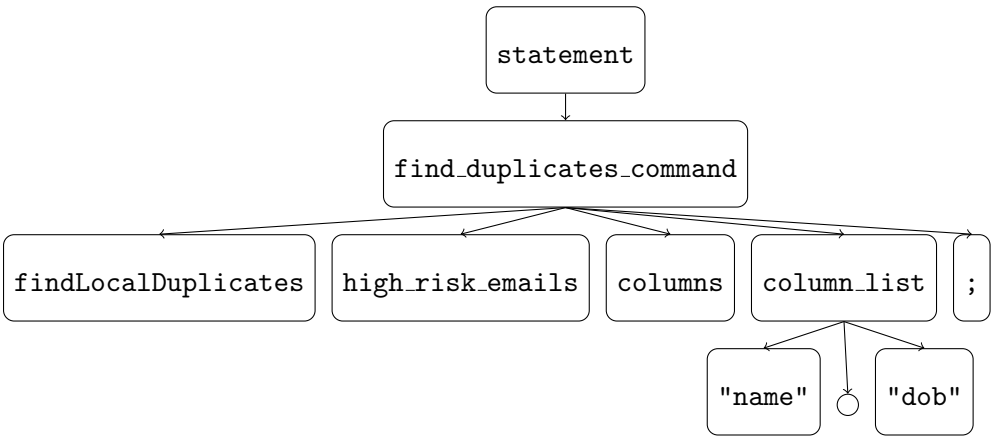

### 2.5.6 output Command

**Example:** `output high_risk_emails -> file "high_risk_emails.csv";`



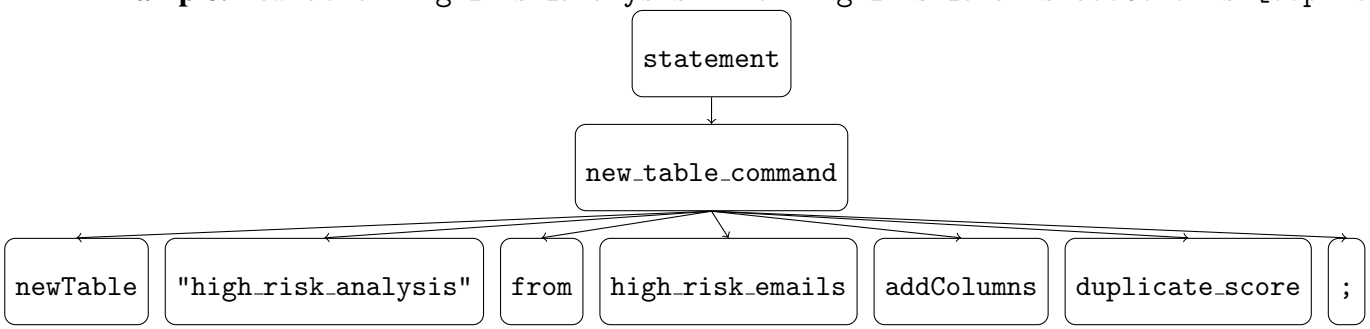

### 2.5.7 findLocalDuplicates Command

**Example:** `findLocalDuplicates high_risk_emails columns ["name","dob"];`




### 2.5.8 newTable Command

**Example:** `newTable "high_risk_analysis" from high_risk_emails addColumns [duplicate_s`




## 2.6 Summary

This report has presented the complete formal grammar and semantic specification of the **Data Quality Validator** DSL. The BNF grammar in Section 2.2 defines an unambiguous, minimal syntax covering database initialisation, column scanning, result filtering, duplicate detection and merging, result export, and table construction. The semantic specification in Section 2.3 assigns a precise meaning to each construct, documents execution rules, and enumerates the error conditions the runtime must detect and report.

The language's design deliberately constrains expressiveness in favour of safety and auditability: destructive operations require explicit opt-in flags, and the expression language is limited to simple threshold comparisons, preventing opaque side effects. These properties make DaQuVa programs straightforward to review, test, and extend.

# 3 DSL Implementation

**3.1 lexer**

**3.2 parser**

**3.3 AST**

**3.4 semantic analysis**

**3.5 execution engine**

# 4 Evaluation & Experiments

## 4.1 Experimant 1

## 4.2 Experimant 2

# 5 Discussion

## 5.1 What was good

## 5.2 What comes to be bad

# Conclusions

Here go your conclusions..